Multiscale Neighborhood Cluster Scene Flow Prior for LiDAR Point Clouds

Jianwang Gan, Guoying Zhang, Juchen Zhang, Yijin Xiong, and Yongqi Gan

***Abstract*— Scene flow estimation, which aims to predict point-wise displacement in 3-D space from sequential data, is a challenging task with wide application in fields such as robotics and autonomous driving. Currently, the accuracy of scene flow estimation from sparse point clouds using prior-based models is suboptimal. Therefore, we revisit the point-by-point scene flow prior and propose a multiscale neighborhood cluster scene flow prior (MNCSFP) to enhance the accuracy of scene flow estimation in sparse point clouds. We optimize the prior model utilizing the multiscale neighborhood cluster feature of the point cloud, in which the point neighborhood is constructed only once. According to the neighborhood index (NI), we design a multiscale neighborhood cluster feature construction (MNCFC) module. The MNCFC module rapidly constructs multiscale neighborhood cluster features using a Gaussian-based neighborhood feature normalization (GNFN) strategy to improve the representation of neighborhood cluster characteristics. Moreover, we propose a neighborhood cluster weighted aggregation (NCWA) module to encode neighborhood cluster features. In NCWA, we design the logsoft function to calculate the neighborhood cluster weights and complete the extraction and aggregation of neighborhood cluster features. Furthermore, we design the multiscale feature fusion (MFF) module that combines the symmetry operation and the logsoft function to complete the fusion of multiscale features to enhance the sparse point feature stability. We evaluate our method on samples from the KITTI, Argoverse, nuScenes, and Waymo Open datasets and demonstrate that it outperforms existing methods and achieves advanced performance.**

***Index Terms*— Logsoft function, multiscale neighborhood cluster, scene flow prior, weighted aggregation.**

I. Introduction

SCENE flow estimation refers to capturing the displacement vector of each point in 3-D space between consecutive frames to provide valuable data for tasks such as semantic segmentation [1], target prediction [2], [3], and 3-D reconstruction [4]. In the past, scholars have utilized images to obtain scene flow [5], [6], [7]. However, image-based methods lack 3-D spatial relationships, leading to inaccurate scene flow estimation for spatial objects. In recent years, the increasing application of 3-D sensors such as LiDAR, especially in autonomous driving and robotics, has advanced the research on 3-D point cloud scene flow estimation. Currently, scene flow estimation is divided into three categories: supervised scene flow estimation models, self-supervised scene flow estimation models, and prior-based scene flow estimation models.

The supervised scene flow estimation methods [8], [9], [10] commonly employ consecutive frames from synthetic datasets like FlyingThing3D [11] to train models in a supervised manner. After model training, researchers fine-tune it on real-world datasets such as KITTI [12], [13] to enhance its adaptability to actual scenarios. Although supervised learning-based scene flow estimation methods exhibit impressive accuracy, the labeling process for point-by-point displacement vectors in consecutive frames of real-world scenarios is both complex and time-consuming, which poses a limitation on its widespread application. In contrast, self-supervised learning-based scene flow estimation models offer two strategies for estimating scene flow. The first self-supervised model [14], [15] is based on the concept of regression, aiming to predict scene flow information accurately in three dimensions. However, given the unordered and irregular nature of point clouds, this approach necessitates a large amount of data for the model to converge. The second self-supervised approach [16], [17] utilizes information from two consecutive frames of point clouds to estimate the motion of each point from the current to the next frame. In this method, existing convex optimization techniques are utilized to learn point features, thereby avoiding the complexity of dealing with 3-D spatial regression problems and significantly simplifying the task of scene flow estimation. However, as depicted in Fig. 1, both supervised and self-supervised approaches require a substantial amount of data for model training, often resulting in limited performance in terms of generalization and scene transferability.

To enhance the accuracy and versatility of scene flow estimation, scholars have introduced a model that leverages prior knowledge to reduce its dependency on specific datasets. This innovative model [18], [19], [20] effectively utilizes prior knowledge to construct optimization strategies, thereby reducing the model's reliance on training datasets. As illustrated in Fig. 1, prior-based scene flow estimation methods primarily rely on information from adjacent sequential frames, rather than large-scale training data. Li et al. [19] pioneered the neural scene flow prior (NSFP). The NSFP constructs a loss function using Chamfer distance to optimize the model, demonstrating good robustness and the ability to process dense LiDAR points. However, the NSFP has a relatively slow online optimization speed. To address this issue, Li et al. [20]

Received 13 October 2024; accepted 16 December 2024. Date of publication 19 December 2024; date of current version 15 January 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 52204180 and in part by the Fundamental Research Funds for the Central Universities under Grant 2024CXCYZ010. *(Corresponding authors: Guoying Zhang; Yijin Xiong.)*

The authors are with the School of Artificial Intelligence, China University of Mining and Technology (Beijing), Beijing 100083, China (e-mail: bqt2100405027@student.cumtb.edu.cn; zgy@cumtb.edu.cn; 653382371@139.com; bqt2000405025@student.cumtb.edu.cn; bqt2100405028@student.cumtb.edu.cn).

Digital Object Identifier 10.1109/TGRS.2024.3520209

1558-0644 © 2024 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 13:41:52 UTC from IEEE Xplore. Restrictions apply.

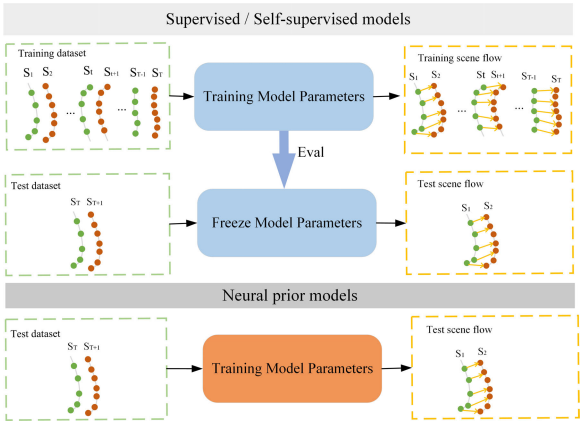


Fig. 1. Supervised or self-supervised models compared to neural prior models. The green point indicates the original point cloud, the orange point indicates the target point cloud, and the yellow arrow indicates the scene flow. Given two consecutive frames of a point cloud, supervised or self-supervised learning methods require data training to calculate network parameters and then freeze the parameters to predict the scene flow. In contrast, the neural prior model requires only a single pair of sequence data, enabling model optimization at runtime. This approach reduces the dependency on large volumes of data, enhancing the model's flexibility and adaptability.

further proposed the fast neural scene flow (FNSF). The NSFP redefines the distance transformation (DT) loss function by eliminating the need for correspondence, significantly boosting the optimization speed of the prior model. The FNSF not only improves computational efficiency but also maintains the advantages of the NSFP in terms of robustness and processing capabilities.

Inspired by prior-based approaches, we identified a lack of prior knowledge about neighborhood scene flow consistency in prior-based models. This absence can lead to inaccurate neighborhood estimations of scene flow in sparse point cloud data, thereby hindering the overall accuracy of scene flow estimation. To fill this gap, we propose a multiscale neighborhood cluster scene flow prior (MNCSFP), a scene flow estimation model incorporating neighborhood consistency prior. Specifically, our main contributions are as follows.

1) To tackle the problem of inaccurate scene flow estimation in sparse point cloud data, we introduce the concept of neighborhood consistency prior knowledge. To better represent point-wise neighborhood features, we propose a Gaussian-based neighborhood feature normalization (GNFN) operation. Using the GNFN operation, we have designed a uniformly sampled multiscale neighborhood clustering feature construction (MNCFC) module to efficiently construct multiscale neighborhood cluster features.
2) To achieve a stable representation of sparse point features, we propose the logsoft function to aggregate multiscale neighborhood cluster features. Based on the logsoft function, we have designed a weight-sharing neighborhood cluster weighted aggregation (NCWA) module to aggregate neighborhood cluster features. Additionally, to fuse multiscale features, we utilize the logsoft function to design a multiscale feature fusion (MFF) module, enabling MFF.
3) After completing the design of each module, we constructed the multiscale neighborhood clustering scene flow prior model. During the optimization process, we combined the correspondence-free loss with our neighborhood smoothness regularization to optimize the MNCSFP, aiming to improve the accuracy and neighborhood consistency of scene flow estimation in sparse point clouds.

To comprehensively validate the generalizability and stability of the MNCSFP model, we conducted experiments on both dense 64-line LiDAR datasets (KITTI [12], [13] and Waymo [21] datasets) and sparse 32-line LiDAR datasets (Argoverse [22] and nuScenes [23] datasets). The experimental results demonstrate that on the dense 64-line LiDAR datasets, our MNCSFP model achieves a performance comparable to existing advanced methods; on the sparse 32-line LiDAR datasets, the MNCSFP model realizes a significant increase in accuracy.

II. Related Work

Early research [24], [25], [26], [27], [28] efforts were primarily based on the assumption of rigid body motions within scenes, aiming to enhance the performance of the iterative closest point (ICP) [29] algorithm. However, in the real world, where static and dynamic objects frequently coexist, relying exclusively on global transformation parameters proves inadequate for accurately capturing the displacement and direction changes of dynamic objects. With the continuous emergence of LiDAR point cloud datasets like KITTI [12], [13] and nuScenes [23], researchers have pivoted their focus toward deep learning models. Presently, the arena of deep learning models for scene flow estimation is divided into three primary categories: supervised scene flow estimation models, self-supervised scene flow estimation models, and prior-based scene flow estimation models.

A. Supervised-Based Scene Flow Estimation

In the supervised learning domain for scene flow estimation, Liu et al. [9] introduced FlowNet3D. This novel model leverages PointNet++ [30] for encoding point cloud features and incorporates a flow embedding module to fuse features between frames. This pioneering work inspired subsequent research efforts such as FlowNet3D++ [10] and HPLFlowNet [8]. Puy et al. [16] proposed FLOT, which initiates flow estimation through graph matching and then refines features using convolution and regression. FlowStep3D, proposed by Kittenplon et al. [17], adopts a recursive architecture that iteratively aligns point clouds to improve scene flow predictions, showcasing its potential in handling complex dynamic scenes. Moreover, there are weakly supervised methods such as the one introduced by Gojcic et al. [31], which combines 3-D scene flow estimation with foreground segmentation to infer object flow information. Although these supervised methods have achieved remarkable results on synthetic datasets, their generalizability in real-world scenarios is limited due to the complex and time-consuming process of annotating scene flow, restricting the practical application of supervised models.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 13:41:52 UTC from IEEE Xplore. Restrictions apply.

B. Self-Supervised Scene Flow Estimation

In the domain of self-supervised estimation models, Baur et al. [32] introduced a network for self-supervised scene flow estimation that utilizes robust rigid self-motion estimation. Their approach predicts motion segmentation and estimates scene flow. Mittal et al. [15] proposed the JustGoFlow method, which leverages nearest neighbors and cycle consistency principles to design two loss functions for model optimization. Tishchenko et al. [33] differentiated scene flow into nonrigid flow and self-motion flow, iterating to refine rigid transformations and then optimizing the nonrigid flow from the subtracted rigid component. Wu et al. [34] developed the PointPWC-Net model, which can be trained using supervised or self-supervised approaches. PointPWC-Net employs a coarse-to-fine strategy to directly process 3-D point cloud scenes with significant motion. Lang et al. [35] proposed the SCOOP model, which initializes flow as soft correspondences between consecutive frame point clouds and trains a pure correspondence model. In self-supervised learning, various models typically require extensive training on large datasets, a process that is time-consuming and results in models with limited scene transferability.

C. Scene Flow Estimation Based on Prior

In the field of prior-based scene flow estimation, researchers have developed optimization strategies that utilize prior knowledge to acquire scene flow, thereby eliminating the need for training on large datasets. Pontes et al. [18] proposed an interpretable objective function that incorporates graph Laplacian regularization for point cloud scene flows, making it suitable for self-supervised scene flow optimization and runtime flow estimation. Li et al. [19] introduced the NSFP, which constructs its prior model using a multilayer perceptron (MLP). This model implicitly regularizes the current frame with the point cloud from the previous frame, estimating the scene flow for each point in the current frame. Li et al. [20] proposed the FNSF to enhance the optimization speed of the NSFP. They identified that the computational bottleneck in the NSFP lies in the loss function rather than the network structure. To address this issue, they reevaluated the DT and employed a loss function without correspondences to accelerate the runtime optimization process.

III. Methodology

Inspired by prior-based scene flow estimation models, we propose a novel neighborhood consistency scene flow prior model and a corresponding optimization strategy. To clarify our approach, we will detail the following key aspects: 1) problem redefinition and optimization; 2) MNCSFP model; and 3) scene flow prior optimization.

A. Problem Redefinition and Optimization

1) Problem Redefinition: In a dynamic scene, point clouds S_t and S_{t+1} are captured at moments t and $t+1$, respectively, where the number of points in S_t is denoted as $|S_t|$ and the number of points in S_{t+1} is denoted as $|S_{t+1}|$. Generally, $|S_t| \neq |S_{t+1}|$, and the point clouds are represented as sets without point correspondences. We extract the point cloud neighborhood information S_t^n, where n represents the number of neighboring points in S_t. For any arbitrary 3-D point $p \in S_t$, we select a neighborhood feature $p^n \in S_t^n$ associated with p. The translation vector (or flow vector) $f \in R^3$ can model the movement from time t to time $t+1$

$$\hat{p} = p + f = p + h\left(p, p_i^n\right) \tag{1}$$

where the prediction of the scene flow from S_t to S_{t+1} is denoted $\hat{p}$ and $p_i^n \in p^n$, respectively, $h(\cdot)$ is a function with the current point p, and the neighborhood point p^n as inputs. The neighborhood representation of point p is constructed using $h(\cdot)$ to obtain more stable single-point features. The set of flow vectors for all 3-D points constitutes the scene flow $F = \{f_i\}_{i=1}^{|S_t|}$.

2) Optimization: To obtain the optimal scene flow F, we introduce a local smoothing term in addition to minimizing the distance between the point cloud neighborhood information at time t (S_t) and time $t+1$ (S_{t+1}). This local smoothing term helps enforce smoothness in the estimated motion field. The optimization problem for the scene flow can be redefined as follows:

$$\hat{F} = \underset{F}{\operatorname{argmin}} \sum_{p \in S_t,\, p^n \in S_t^n} D(p + h\left(p, p^n\right), S_{t+1}) + \lambda L_{\text{smooth}} \tag{2}$$

$$h(p, p^n) = h\left(\sum_{i=1}^{n} w_i^n C(p, p_i^n)\right) \tag{3}$$

where the input of $h(\cdot)$ is a weighted sum of neighborhood features and current features to increase the point p feature representation. $C(\cdot)$ is the neighborhood feature mapping and $w^k \in W_t^k$ are the neighborhood feature weights. $D(\cdot)$ is used to measure the nearest point between $p + h(\cdot)$ and S_{t+1}. To ensure the smoothness of the scene flow within the neighborhood, we introduce a regularization term called L_{smooth}. This term imposes constraints on the neighborhood flow, encouraging neighboring points to have consistent motion patterns. The weight of the regularization term is denoted by λ. We propose an a priori algorithm for neighborhood cluster-consistent neural network models to regularize the scene flow further. Neighborhood cluster features can capture a stable and smooth scene flow, making the estimation results more accurate and robust.

B. Multiscale Neighborhood Cluster Scene Flow Prior Model

We propose an MNCSFP model, which only requires the source point cloud (S_t) and the target point cloud (S_{t+1}) between two consecutive frames for optimization. The MNCSFP model is illustrated in Fig. 2. First, this model constructs a neighborhood index (NI) of S_t using K-nearest neighbors (KNNs) operation only once, which facilitates subsequent GNFN operations and neighborhood consistency prior optimization. Following this, we sequentially implement the multiscale neighborhood cluster feature construction (MNCFC), NCWA, MFF, and point feature encoding (PFE)

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 13:41:52 UTC from IEEE Xplore. Restrictions apply.

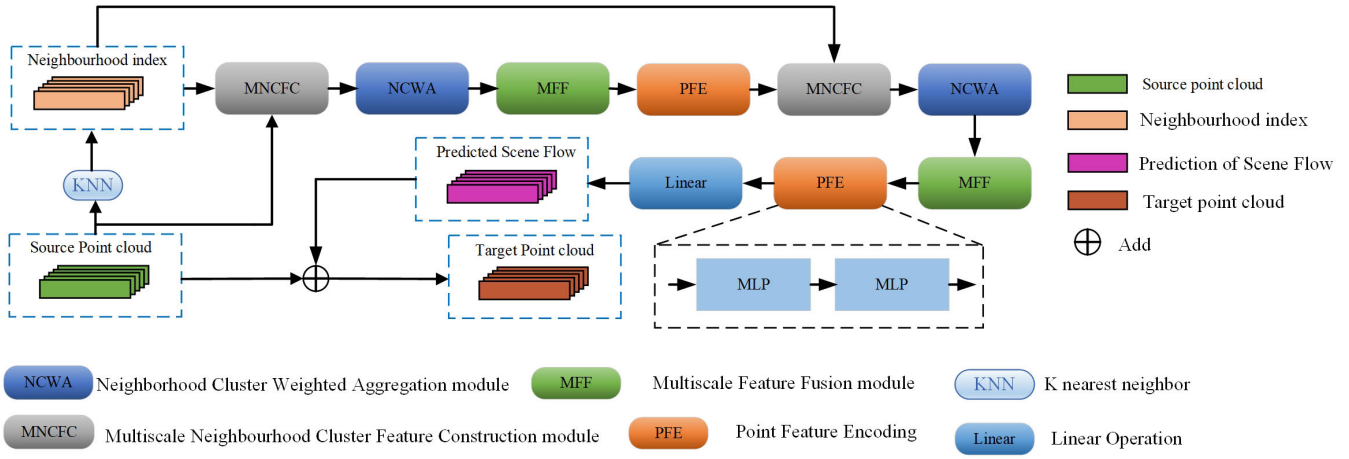


Fig. 2. MNCSFP model. The MNCSFP network consists of sequentially the MNCFC, NCWA, MFF, and PFE modules, where the neighborhood is obtained by KNN operation (used only once). We designed MNCFC to construct neighborhood cluster features at each scale, NCWA for extracting and aggregating neighborhood cluster features, and MFF for fusing multiscale neighborhood features. The PFE consists of an MLP, and the experimental MLP is implemented by linear operations and the ReLU activation function.

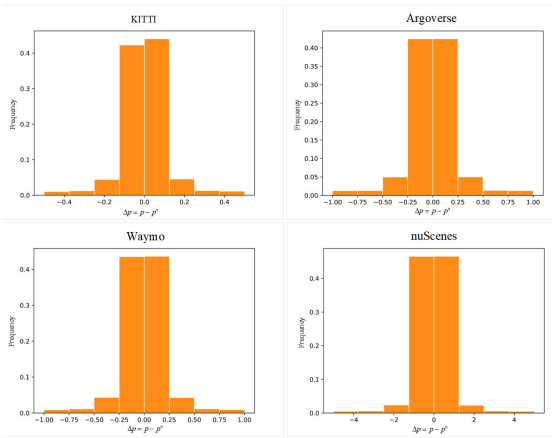


Fig. 3. Distribution statistics of point cloud neighborhood differences. In each chart, the horizontal axis $\triangle p = p - p^n$ represents the difference between any point p in the point cloud and its neighborhood point p^n. The vertical axis indicates the frequency of $\triangle p$ in a specified range. The statistical charts exhibit notable similarities across diverse datasets.

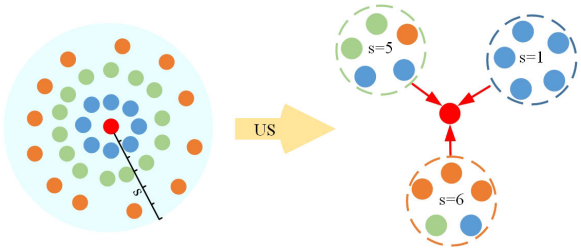


Fig. 4. US to construct multiscale neighborhood cluster features. The red point represents the feature of the present point, while the blue, green, and orange points indicate the nearest, second nearest, and farther points within the neighborhood of the red point, respectively. During the US process, the number of sampling points is set to 5. As the sampling step size s increases, the process of sampling transitions from capturing the nearest neighbor ($s = 1$) solely to encompassing progressively more distant neighborhood points ($s = 6$), thereby facilitating the construction of multiscale neighborhood cluster features.

modules. These modules are designed to build and integrate multiscale neighborhood features effectively. After encoding the neighborhood features, our scene flow $F = \{f_i\}_{i=1}^{|S_t|}$ is computed using a linear operation. Next, we will discuss the necessity of the GNFN operation and explain our proposed core modules: MNCFC, NCWA, and MFF.

1) Gaussian-Based Neighborhood Feature Normalization: The neighborhood features of point cloud data are conducive to the encoding of local geometric information and spatial relationships [30], [36], [37], [38], [39], which can effectively improve the representation of single-point features. To construct a better point cloud neighborhood feature quantification strategy, we analyzed the KITTI [12], [13], Argoverse [22], Waymo [21], and nuScenes [23] datasets, counting their neighborhood difference information distribution as shown in Fig. 3. These neighboring points are efficiently aggregated through the KNN algorithm. By observing Fig. 3, we notice that the frequency distribution of neighborhood difference features $\triangle p = p - p^n$ in various datasets exhibits a Gaussian distribution shape, and the standard deviations of each dataset show differences. These discoveries have led us to propose a GNFN operation, which aims to improve the encoding of neighborhood information and further promote the optimization process of the network. Specifically, the encoding of point neighborhood features can be described as follows:

$$G(p, p^n) = \text{Concat}\left(p, p^n, \frac{(p - p^n) - \mu}{\delta + \varsigma}\right) \tag{4}$$

$$\mu = \frac{1}{n}\sum_{i=1}^{n}(p - p_i^n) \tag{5}$$

$$\delta = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left((p - p_i^n) - \mu\right)^2} \tag{6}$$

where $p_i^n \in p^n = \{p_1^n, \ldots, p_i^n | i = 1, \ldots, n\}$, μ represents the neighborhood mean, δ represents the neighborhood variance, and ς is a small term set to $1e^{-10}$ to prevent division by zero. The normalization operation in (4) can effectively standardize data representation and reduce the impact of outlier samples on model optimization.

2) MNCFC Module: The MNCFC module effectively encodes neighborhood cluster features through the GNFN operation, aiming to enhance the stability of neighborhood representations. To address the issue of increased time and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 13:41:52 UTC from IEEE Xplore. Restrictions apply.

spatial complexity caused by a higher number of points in neighborhood clusters, we skillfully introduce uniform sampling (US) techniques to construct multiscale neighborhood clustering features. As illustrated in Fig. 4, we initially use the KNN algorithm to swiftly search for the neighborhood of each point, setting the number of neighborhood points to 32 to ensure neighborhood completeness. During the US phase, we fix the number of sampling points at K and flexibly control the sampling range by adjusting the step size s, thereby capturing multiscale neighborhood cluster features while maintaining computational efficiency. This approach not only optimizes the use of computational resources but also improves the local structural representation of point cloud data.

Specifically, the MNCFC process is shown in Fig. 5. Given a point set feature $P \in R^{N\times C}$, where N denotes the number of points and C is the number of features. We obtain the neighborhood of each point based on the NI. Take any point feature $p_i \in P$, i is the point index, using i can find the neighborhood cluster in NI, and the neighborhood of p_i can be denoted as $p_i^n = \{p_{i,1}, \ldots, p_{i,j}, \ldots, p_{i,n} | j = 1, \ldots, n\}$. After obtaining the n neighborhood points of p_i, we design the neighborhood US to mitigate the computational increase caused by neighborhood expansion. We sample the K neighborhood points in p_i^n at step size s encoded as $p_i^s = \{p_{i,1}, \ldots, p_{i,j\times s}, \ldots, p_{i,j\times K} | j = 1, \ldots, K\} \subset p_i^n$. We show the coded arbitrary point features as follows:

$$g_i^s = \text{Concat}\left(p_i, p_i^s, \frac{(p_i - p_i^s) - \mu_s}{\sigma_s + \zeta}\right) \tag{7}$$

where Concat$(\cdot)$ denotes the concatenation operation; $g_i^s \in R^{K\times C}$ denotes the point feature p_i, the neighborhood cluster feature p_i^s, and the neighborhood cluster normalization feature $((p_i - p_i^s) - \mu_s)/(\sigma_s + \zeta)$ concatenated to enhance the feature representation of the point. μ_s and σ_s denote the neighborhood cluster mean and variance of the sample points at different scales, respectively. In MNCFC, we take the number of neighborhood points $K = 5$ and the sampling step $s = \{1, 5, 6\}$ for constructing different neighborhood features.

3) NCWA Module : The feature extraction and aggregation process of our NCWA module is illustrated in Fig. 5. After obtaining the point neighborhood cluster coding features $G^s = \{g_1^s, \ldots, g_i^s, \ldots, g_N^s\} \in R^{N\times K\times C_{\text{in}}}$, the features are extracted using MLP, and this operation is calculated as shown in the following equation:

$$H^s = \text{MLP}\left(G^s; \Theta^s\right) \tag{8}$$

where MLP$(\cdot)$ denotes the execution of MLP operations along the channel dimension and Θ^s is the trainable parameter. $H^s = \{h_1^s, \ldots, h_i^s, \ldots, h_N^s\} \in R^{N\times K\times C_{\text{out}}}$ denotes the features extracted from multiscale neighborhood clusters. $h_i^s = \{h_{i,1}^s, \ldots, h_{i,j}^s, \ldots, h_{i,K}^s | j = 1, 2, \ldots, K\} \in R^{K\times C_{\text{out}}}$, K denotes the number of neighborhood points, and C_{out} denotes the number of output channels.

After obtaining the feature H^s, we designed a weight-sharing module and a logsoft function to aggregate the neighborhood clustering features. We constructed a shared MLP to learn multiscale neighborhood cluster features and calculated the weights of neighborhood clusters through our logsoft function to realize the weighted aggregation of clustering features. The logsoft function is shown in Fig. 6. We compared the commonly used weight calculation function, Softmax, with our logsoft function. Due to the small number of sampling points in the neighborhood cluster (i.e., $K = 5$), there is less redundant information. In this case, when using the Softmax function to calculate the feature weights of the neighborhood cluster along dimension K, the exponential mapping in the Softmax function will significantly amplify the differences between features, which may ignore the features of relatively distant neighboring points. This amplification effect leads to a reduction in the diversity of the aggregated features, which, in turn, affects the stability of scene flow estimation. In contrast, our logsoft function has a gentle trend, which can maintain feature differences while reducing the influence of extreme values. This characteristic is beneficial for representing the diverse features of the neighborhood cluster, further enhancing the stability and accuracy of those features. The formula for the logsoft function is provided below

$$\text{logsoft}(x) = \frac{\log(1 + e^x)}{\sum_K \log(1 + e^x)} \tag{9}$$

where the logsoft$(\cdot)$ function calculates the neighborhood cluster weights along dimension K.

To alleviate the spatiotemporal complexity, we also designed the weight-shared MLP (shared MLP) to learn the neighborhood cluster weights, which are calculated as shown in the following equation:

$$W^s = \text{logsoft}\big(\text{MLP}\big(H^s; \Theta\big)\big). \tag{10}$$

In the above equation, $W^s = \{w_1^s, \ldots, w_i^s, \ldots, w_N^s\} \in R^{N\times K\times C_{\text{out}}}$ denotes the output cluster weights for each neighborhood, $w_i^s = \{w_{i,1}^s, \ldots, w_{i,j}^s, \ldots, w_{i,K}^s | j = 1, 2, \ldots, K\} \in R^{K\times C_{\text{out}}}$. The training weights Θ are shared by different scale features.

The learned weights W^s can adaptively select neighborhood cluster features to describe point features. Formally, we define the neighborhood cluster feature weighted aggregation operation as follows:

$$O^s = \sum_K W^s \odot H^s \tag{11}$$

$$o_i^s = \sum_K w_i^s \cdot h_i^s = \sum_{j=1}^{K} w_{i,j}^s \cdot h_{i,j}^s \tag{12}$$

where $O^s = \{o_1^s, \ldots, o_i^s, \ldots, o_N^s\} \in R^{N\times C_{\text{out}}}$ and O^s aggregates the neighborhood cluster features to obtain more stable features. Then, we concatenate the aggregated features at each scale, as shown in the following equation:

$$Q = \text{Concat}\big(O^{s_1}, \ldots, O^{s_m}\big) \tag{13}$$

where the set of sampling steps $s = \{s_1, \ldots, s_m\}$, m is the number of elements of the set s, and $Q \in R^{N\times m\times C_{\text{out}}}$.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 13:41:52 UTC from IEEE Xplore. Restrictions apply.

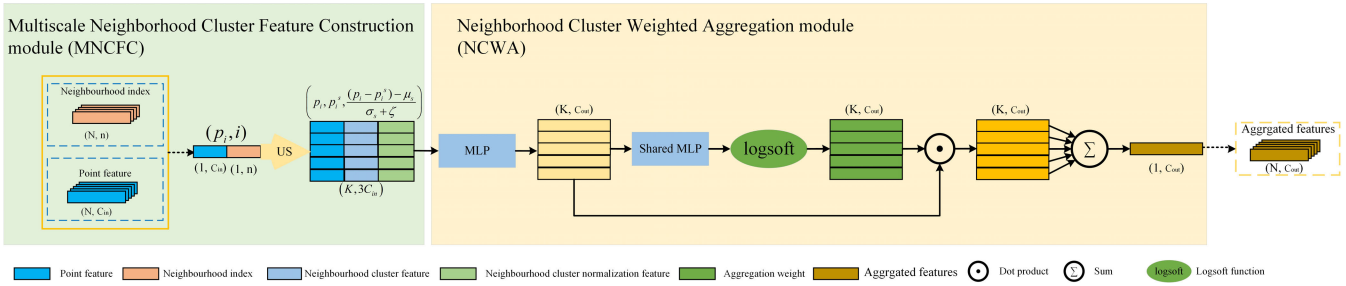


Fig. 5. Proposed MNCFC module and the NCWA module. The NI and point features are input, and the neighborhood features are constructed using our proposed MNCFC. Subsequently, stable point features are obtained by aggregating neighborhood cluster features using our proposed NCWA.

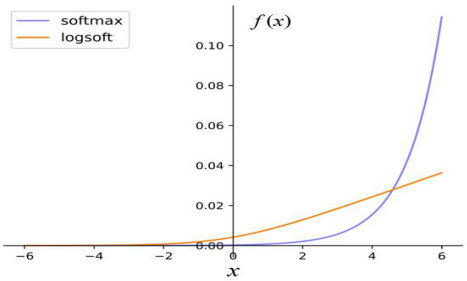


Fig. 6. Comparative analysis of logsoft and softmax function characteristics. This figure showcases the disparities between our logsoft function and the conventional softmax function when processing incoming features. The softmax function amplifies inputs through exponential calculations, thereby rendering it highly responsive to the maximum value in the input. In contrast, our logsoft function guarantees a smoother transition from input features to outputs.

4) MFF Module: To robustly represent the multiscale features in Q, MFF employs a common symmetry operation in point clouds, that is, taking the max and mean values of the multiscale features to fuse different features. The symmetric features are fused using the proposed logsoft function weighting. The MFF is shown in Fig. 7. Given the input feature Q, we first perform max-pooling and mean pooling along the m-dimension to concatenate the pooling results. Then, the weights are computed using our proposed logsoft to fuse the symmetric features. The calculation formula is as follows:

$$\mathrm{MF}(Q) = \mathrm{logsoft}(\mathrm{MLP}(\mathrm{MP}(Q); \Theta_M)) \tag{14}$$

$$\mathrm{MP}(Q) = \mathrm{Concat}(\mathrm{Max}\,P(Q), \mathrm{Mean}\,P(Q)) \tag{15}$$

where $\mathrm{MP}(Q) \in R^{N\times 2\times C_{\mathrm{out}}}$, $\mathrm{MF}(Q) \in R^{N\times C_{\mathrm{out}}}$, and Θ_M denote the learnable parameters. In short, given an input point cloud P, our multiscale neighborhood feature encoding module and aggregation module learn the geometric patterns and features of different neighborhood points to generate robust feature information $\mathrm{MF}(Q)$.

After the locally weighted feature aggregation, each point in the point cloud incorporates its features and rich information from its neighborhood. Therefore, in the PFE module, we employ a two-layer MLP operation to extract multiscale fused features to provide reliable inputs for network prediction. Our MNCSFP model is sequentially constructed from MNCFC, NCWA, MFF, and PFE modules, which effectively aggregates neighborhood cluster features and fuses multiscale features. Our proposed MNCSFP captures more robust point features and significantly improves the accuracy of scene flow estimation.

C. Scene Flow Prior Optimization

We have integrated two regularization terms to optimize the MNCSFP model, detailed in (2). To enhance clarity in the description, we introduce a parameter, Γ, which is the parameter to be optimized. Equation (2) can be reformulated as follows:

$$\Gamma' = \underset{\Gamma}{\mathrm{argmin}} \sum_{p\in S_t, p^k\in S_t^k} D\big(p + H(p, p^n, \Gamma), S_{t+1}\big) + \lambda L_{\mathrm{smooth}}(H(p, p^n, \Gamma)) \tag{16}$$

where the scene flow $f = H(p, p^n, \Gamma)$ represents the output of the network, and the optimization converges to $f' = H(p, p^n, \Gamma')$. Directly calculating the distance from $p + f'$ to each point in the set S_{t+1} would result in a high computational cost for $D(\cdot)$. To expedite the computation of $D(\cdot)$, we use the DT introduced in FNSF [20] to construct the loss function. In addition, to enhance the consistency of scene flows within the neighborhood, a neighborhood smoothing regularization term $L_{\mathrm{smooth}}(\cdot)$ is designed to improve the accuracy and robustness of scene flow estimation. Next, we will elaborate on the calculation methods of these two regularization terms.

1) DT Regularization Term: Given two consecutive frame point clouds S_t and S_{t+1}, as well as scene flow F computed by a prior model. To avoid the time consumption of establishing point-by-point correspondences between frames as in NSFP, Li et al. [20] only used the target point cloud S_{t+1} and the spatial coordinates ranges of both frame point clouds to construct a DT mapping G. This approach facilitates the computation of the minimum distance between the predicted value, $y \in S' = \mathrm{norm}(S_t + F)$, and a regular spatial point, $q \in G$, allowing for a swift optimization of the scene flow. Here, the $\mathrm{norm}(\cdot)$ function is a regularization function that normalizes the predicted values to the range $[-1, 1]$ and q represents a regular spatial point in the voxel G. Specifically, the loss function is computed as follows:

$$L_{\mathrm{DT}}(S_t, S_{t+1}, F) = \frac{1}{|S_t|}\sum \mathrm{DT}(S_t, S_{t+1}, F) \tag{17}$$

$$\mathrm{DT}(S_t, S_{t+1}, F) = \min_{y\in S', q\in G} \|y - q\|_2. \tag{18}$$

The DT of points can be obtained by calculating the minimum distance from each point, $y \in S'$, to the vertices q of the DT map G. Unlike the construction of the optimization term in

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 13:41:52 UTC from IEEE Xplore. Restrictions apply.

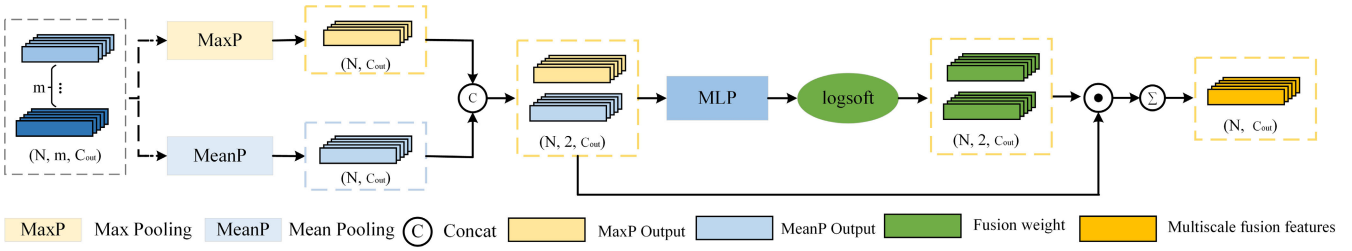


Fig. 7. MFF module. We use maximum and mean pooling to fuse multiscale features and concatenate the extracted symmetry features. The symmetric features are then aggregated using our proposed logsoft weighted aggregation to obtain more stable point features.

the NSFP [19], (18) does not require point correspondence searching, allowing for faster optimization.

2) Smooth Regularization Term: To maintain consistency in the scene flow within the neighborhood of each point in the source point cloud (S_t), we introduce a smoothing regularity term into the optimization of the proposed prior model. The objective of this term is to encourage smoothness in the predicted scene flow values. In other words, neighboring points should have similar scene flow predictions. The smoothing loss is defined as follows:

$$L_{\text{smooth}} = \frac{1}{|S_t|n} \sum_{f \in F} \sum_{l \in N_f} \|f - f_l\|_1 \tag{19}$$

where f is the point-by-point scene flow prediction in S_t and N_f is the k-NI of each point in S_t, obtained by the first KNN calculation. f_l is the neighborhood flow of scene flow f. The overall loss calculation formula is expressed as follows:

$$L_{\text{total}} = L_{\text{DT}} + \lambda L_{\text{smooth}}. \tag{20}$$

The hyperparameter λ balances the contribution of different loss terms in L_{total}. In this context, the optimal value for λ is 1.5.

IV. Experiments

To comprehensively evaluate our MNCSFP model, we conducted experiments on the KITTI [12], [13], Argoverse [22], nuScenes [23], and Waymo Open [21] datasets. All experiments were carried out using a single NVIDIA RTX 3090Ti GPU. For a clear description of the experimental content, we have organized our experiments into six main parts: 1) datasets and metrics; 2) implementation details; 3) quantitative analysis; 4) qualitative analysis; 5) ablation study; and 6) application. Next, we will elaborate on these six parts in detail.

A. Datasets and Metrics

1) Datasets: We conducted experiments on four real-world scenario datasets to evaluate the performance of our MNCSFP method.

1) *KITTI [12], [13]:* The dataset was collected using a 64-line LiDAR, which provides dense point clouds and rich 3-D information. It is not only one of the earliest autonomous driving datasets but also continues to be widely used in various autonomous driving-related research due to its comprehensiveness and diversity. To ensure a fair comparison, our test set is the scene flow annotated subset in FlowNet3D [9].
2) *Argoverse [22]:* The Argoverse dataset was collected using a 32-line LiDAR and contains a wealth of dynamic objects. Compared to LiDARs with higher line counts, it generates sparser point clouds. Although it does not provide official scenario stream annotations, we followed the approach described in [18] and [19] to annotate the scenario streams. We removed ground points, separated rigid and nonrigid objects using the provided target annotations, and computed point-by-point flow vectors based on the car and object poses. These translation vectors were then combined to generate point-by-point scene flow annotations.
3) *nuScenes [23]:* Similar to Argoverse, the nuScenes dataset is also captured using a 32-line LiDAR, resulting in sparse point cloud data. As the scene flow annotation is not officially provided, we applied the same annotation procedure described for Argoverse.
4) *Waymo Open [21]:* The Waymo Open dataset was collected using a 64-line LiDAR, resulting in dense point cloud data. Since the Waymo Open dataset does not provide scene flow annotations, we obtained pseudo-scene flow labels using existing annotation information. The annotation process employs the methods proposed in [40] and [41].

By evaluating our MNCSFP method on these diverse datasets, we ensure the robustness and effectiveness of our approach across various real-world scenarios.

2) Metrics: We employed the same evaluation metrics as [19] and [20], which consist of four metrics.

1) *3-D Endpoint Error (3DEPE):* This metric measures the average point-to-point distance between the predicted scene flow (F_{pred}) and the ground-truth scene flow (F_{gt}). It can be expressed as $\sum \|F_{\text{pred}} - F_{\text{gt}}\|_2 / |S_t|$
2) *Strict Accuracy (Acc5):* This metric calculates the percentage of point pairs where the absolute difference between F_{pred} and F_{gt} is less than 0.05 m, or the relative point error $\|F_{\text{pred}} - F_{\text{gt}}\|_2 / F_{\text{gt}} < 5\%$.
3) *Relaxed Accuracy (Acc10):* Similar to the strict accuracy metric, this metric calculates the percentage of point pairs where the absolute difference between F_{pred} and F_{gt} is less than 0.1 m, or the relative point error $\|F_{\text{pred}} - F_{\text{gt}}\|_2 / F_{\text{gt}} < 10\%$.
4) *Angle Error (AE):* This metric measures the average angular difference between F_{pred} and F_{gt}.

B. Implementation Details

Our proposed MNCSFP model is shown in Fig. 2. The model consists of MNCFC, NCWA, and MFF proposed in

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 13:41:52 UTC from IEEE Xplore. Restrictions apply.

TABLE I
QUANTITATIVE EVALUATION OF OUR METHOD AND OTHER GOOD-PERFORMING METHODS ON THE KITTI AND ARGOVERSE DATASETS

Number of Points	Methods	KITTI Scene Flow [12], [13]				Argoverse Scene Flow [22]			
		$3DEPE\downarrow$	$Acc5(\%)\uparrow$	$Acc10(\%)\uparrow$	$AE\downarrow$	$3DEPE\downarrow$	$Acc5\,(\%)\uparrow$	$Acc10(\%)\uparrow$	$AE\downarrow$
2048 pts	• FlowNet3D [9]	0.199	10.44	38.89	0.386	0.455	1.34	6.12	0.736
	• PointPWC-Net [34]	0.142	29.91	59.83	0.239	0.405	8.25	25.47	0.674
	• JustGoFlow [15]	0.218	10.17	34.38	0.254	0.524	8.80	20.28	0.715
	• PointPWC-Net($self$) [34]	0.177	13.29	42.15	0.272	0.409	9.79	29.31	0.643
	• Non-rigid ICP [42]	0.338	22.06	43.03	0.460	0.461	4.27	13.90	0.741
	• Graph prior [18]	0.099	63.60	81.18	0.176	0.257	25.24	47.60	0.467
	• NSFP [19]	0.05	81.68	93.19	0.133	0.159	38.43	63.08	0.374
	• FNSF [20]	0.173	39.27	56.54	0.286	0.278	21.2	43.52	0.499
	• MNCSFP(Ours)	**0.034**	**95.6**	**97.9**	**0.103**	**0.092**	**67.67**	**83.12**	**0.354**
8192 pts	• FLOT [16]	0.056	75.5	90.8	–	0.821	2.00	8.84	0.967
	• RSDSF [31]	0.094	78.4	88.5	0.280	0.417	32.52	42.52	0.551
	• FlowStep3D [17]	0.177	37.4	66.8	–	0.845	0.01	0.08	1.860
	• NSFP [19]	**0.027**	94.6	96.5	**0.087**	0.113	46.32	72.68	0.347
	• FNSF [20]	0.101	63.19	79.09	0.207	0.118	69.93	83.55	0.352
	• MNCSFP(Ours)	0.042	**96.76**	**97.29**	0.111	**0.055**	**82.86**	**92.41**	**0.285**
All pts	• NSFP [19]	**0.023**	95.40	**98.04**	**0.081**	0.069	71.56	87.80	0.235
	• FNSF [20]	0.0947	75.53	86.0	0.175	0.071	80.05	90.71	0.289
	• MNCSFP(Ours)	0.042	**96.8**	97.07	0.129	**0.043**	**88.81**	**94.79**	**0.235**

this article. The hidden layer cells of each layer are 128. The model's input is a 3-D point cloud S_t, and the output is a point-by-point scene flow collection F. We used the PyTorch [43] framework to implement the network structure, and the MLP operations were implemented using linear operations and the ReLU activation function sequential execution. MNCSFP uses the Adam [44] optimizer to optimize the network. The learning rate is fixed at $8e^{-3}$ throughout the training process. To compute the KNN index, we set the neighborhood size n to 32. This means that we will sample the neighborhood of 32 points and construct neighborhood clusters. For the experiment, we take the neighborhood cluster of points $K = 5$ and the sampling step $s = \{1, 5, 6\}$. To enhance the neighborhood smoothing of the scene flow, we combine the DT regular term and the smoothing regular term for the first time. The loss function's hyperparameter weight λ is set to 1.5 to control the balance between these terms.

C. Quantitative Analysis

Our quantitative analysis results are thoroughly detailed in Tables I and II. In these tables, red dots (•) represent learning-based methods [9], [15], [16], [31], [34] trained on datasets, while blue dots (•) denote prior-based methods [18], [19], [20], [42] that do not depend on training datasets. Additionally, an upward arrow (↑) indicates that higher values correspond to better performance, and a downward arrow (↓) signifies that lower values indicate improved performance.

In Table I, we conducted a comprehensive comparative analysis of the MNCSFP model against various advanced learning-based and prior-based methods. To train the learning-based models, we selected 1000 samples from the 64-line LiDAR collected KITTI dataset for network training or fine-tuning, reserving 50 samples for testing. Furthermore, to further validate the effectiveness of MNCSFP in processing sparse point clouds, we utilized the Argoverse dataset collected with 32-line LiDAR, where 2691 samples were used for training and 212 samples for testing. Observing the results in the "All pts" row of Table I, on the dense KITTI dataset, our MNCSFP model performs comparably to the best NSFP model, both outperforming FNSF. On the sparse Argoverse dataset, our MNCSFP achieves the best results, with an approximate 17% and 8% improvement on the Acc5 metric compared to NSFP and FNSF, respectively. To further validate the stability of our MNCSFP in sparse point cloud data, we sparsified both the KITTI and Argoverse datasets. When randomly sampling to 8192 points, on the KITTI dataset, our MNCSFP model's metrics were comparable to the best NSFP model; on the Argoverse dataset, our MNCSFP achieved the best performance. The random sampling might lead to uneven sparsification of the KITTI point cloud, rendering the performance of our MNCSFP comparable to that of NSFP. Therefore, we proceeded to decrease the point count to 2048. At this sparsity level, we observed that our MNCSFP model achieved optimal performance metrics on both the KITTI and Argoverse datasets.

According to the analysis in Table I, our MNCSFP model exhibits significant advantages in estimating scene flow from sparse point clouds. To further verify the stability and generalization of the MNCSFP model, we conducted additional experiments on both the sparse nuScenes dataset and the dense Waymo Open dataset. To ensure the fairness of the experiments, we selected 310 test samples from the sparse nuScenes dataset and 202 test samples from the dense Waymo Open dataset. As shown in Table II, when all points were utilized for evaluation, our method not only achieved optimal performance but also significant improvements across

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 13:41:52 UTC from IEEE Xplore. Restrictions apply.

TABLE II
QUANTITATIVE EVALUATION OF OUR METHOD AND OTHER GOOD-PERFORMING METHODS ON THE NUSCENES AND WAYMO OPEN DATASETS

Number of Points	Methods	nuScenes Scene Flow [23]				Waymo Open Scene Flow [21]			
		$3DEPE\downarrow$	$Acc5(\%)\uparrow$	$Acc10(\%)\uparrow$	$AE\downarrow$	$3DEPE\downarrow$	$Acc5\,(\%)\uparrow$	$Acc10(\%)\uparrow$	$AE\downarrow$
8192 pts	• FLOT [16]		———			0.702	2.46	11.30	0.808
	• RSDSF [31]		———			0.414	35.47	44.96	0.527
	• NSFP [19]	0.19	38.29	66.46	0.261	0.138	53.62	78.57	0.339
	• FNSF [20]	0.30	41.3	59.7	0.356	0.106	77.53	88.99	0.329
	• MNCSFP(Ours)	**0.119**	**67.73**	**84.89**	**0.185**	**0.062**	**83.24**	**92.94**	**0.290**
All pts	• NSFP [19]	0.195	38.36	66.35	0.262	0.100	76.62	88.56	0.286
	• FNSF [20]	0.292	41.1	61.17	0.329	0.072	84.73	92.24	0.280
	• MNCSFP(Ours)	**0.11**	**67.84**	**85.44**	**0.182**	**0.0589**	**89.21**	**94.28**	**0.255**

all metrics in the sparse nuScenes dataset. By further reducing the point cloud data to 8192 points, we observed notable improvements in metrics on both the nuScenes and Waymo datasets. From the above analysis, it can be inferred that our MNCSFP model has excellent generalizability, effectively improving the stability and accuracy of scene flow estimation in sparse point clouds.

D. Qualitative Analysis

To more intuitively demonstrate the advantages of our MNCSFP model in estimating sparse point cloud scene flow, we conducted a qualitative assessment of the sparse point cloud datasets nuScenes and Argoverse. As shown in Fig. 8, we use a color wheel to represent the magnitude and direction of the scene flow, making the visualization of the scene flow clearer. The color wheel is displayed in the top left corner of the first ground-truth data, with the scene flow colors encoding the size (color intensity) and direction (angle) of the flow vector. Violet-leaning values indicate object movement in a partial northeast direction, red-leaning values indicate southeast movement, green-leaning values indicate southwest movement, and blue-leaning values indicate northwest movement.

In Fig. 8, we display the scene flow estimation results of our MNCSFP model compared to those of the FNSF model. By examining the red checkbox part in Fig. 8(a) from the Argoverse dataset, it can be clearly observed that our MNCSFP model can more completely estimate the scene flow of moving objects. Further inspection of the red checkbox part in Fig. 8(b) from the nuScenes dataset reveals that, in the first row, the MNCSFP model effectively estimates the scene flow of a vehicle moving in the southwest direction (green vehicle). In the second row, the MNCSFP model is more precise and comprehensive in estimating the scene flow of the blue vehicle (moving northwest) and the pink vehicle (moving southeast). These observations further confirm that the neighborhood consistency strategy we proposed can significantly improve the accuracy of scene flow estimation in sparse point cloud data.

Overall, these qualitative results highlight the effectiveness of the proposed method in estimating the scene flow direction. The observed accuracy and consistency with the ground truth affirm the reliability of the predictions.

E. Ablation Study

This section aims to validate the effectiveness of the MNCSFP network module on the KITTI Scene Flow dataset and investigate the impact of different hyperparameters on the loss function. The MNCSFP validation module focuses on several key aspects: the choice of sampling step s in the model, Gaussian-based feature coding of neighborhood clusters, and the validity of the logsoft function. By thoroughly evaluating and analyzing these aspects, we can determine the MNCSFP module's optimal configuration, enhancing the proposed method's performance and reliability across various scenarios.

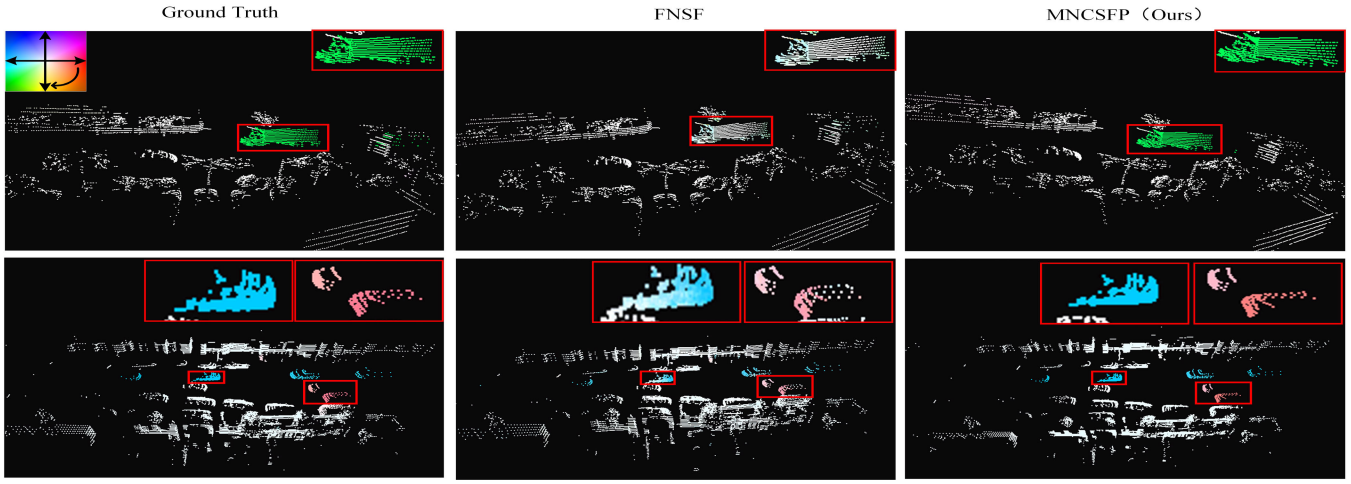
1) Value of the Sampling Step s in MNCSFP: In the MNCFC module, we employ a US strategy to construct multiscale cluster features, thereby reducing the number of neighboring points. This section will explore the impact of the sampling stride set s on the accuracy of scene flow estimation. In our experiments, we utilize sparse point cloud data, obtained through random sampling and comprising 2048 points, as the input for our model. Here, we set the hyperparameter λ in the loss function at 1.5, while keeping other experimental parameters constant. Generally, the greater the number of elements in the sampling stride set s, the greater the number of points sampled, which leads to a more comprehensive representation of multiscale neighborhood features and therefore improves model accuracy. To construct stable neighborhood features with a small number of stride set s elements, we investigated the effects of varying the number of elements and different stride values within the set s on the MNCSFP model.

Table III presents the experimental results for different sampling stride sets s. To stably represent multiscale neighborhood features while reducing the number of points, we initially set s to contain three elements, representing small, medium, and large scales, respectively. Observing the set with three elements, we found that the MNCSFP model performs best when $s = \{1, 5, 6\}$. Further expanding the number of elements in set s to four, we noticed that the MNCSFP model achieves optimal results with $s = \{1, 4, 5, 6\}$. A comparison of the performance metrics for $s = \{1, 5, 6\}$ and $s = \{1, 4, 5, 6\}$ revealed only minor numerical differences, indicating closely matched performances between the two

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 13:41:52 UTC from IEEE Xplore. Restrictions apply.



(a)



(b)

Fig. 8. Our proposed MNCSFP method was evaluated qualitatively on the Argoverse and nuScenes scene flow datasets. Red boxes indicate comparisons with the FNSF method. In the visual assessment, the top left corner of the first ground-truth data represents a color wheel. The magnitude and direction of the scene flow values correspond to the color and direction of the color wheel, respectively. This visualization helps to understand the difference in spatial distribution between the ground truth and the predicted scene flow. (a) Qualitative results for the Argoverse dataset sample. (b) Qualitative results for the nuScenes dataset sample.

TABLE III

EFFECT OF SAMPLING STEP SIZE s ON MNCSFP

s	$3DEPE\downarrow$	$Acc5(\%)\uparrow$	$Acc10(\%)\uparrow$	$AE\downarrow$
$\{1,2,3\}$	0.04	93.26	95.7	0.129
$\{1,3,4\}$	0.038	94.7	96.58	0.106
$\{1,4,5\}$	0.039	94.6	96.84	0.125
$\{1,5,6\}$	**0.034**	**95.6**	**97.99**	**0.103**
$\{1,6,7\}$	0.05	94.7	96.25	0.127
$\{1,3,5,6\}$	0.039	94.5	96.8	0.098
$\{1,4,5,6\}$	**0.027**	**96.2**	**98.4**	**0.095**
$\{1,5,6,7\}$	0.027	96.1	98.4	0.096

TABLE IV

IMPACT OF NORMALIZATION IN POINT NEIGHBORHOOD CHARACTERIZATION

Points	$\triangle p$	$3DEPE\downarrow$	$Acc5(\%)\uparrow$	$Acc10(\%)\uparrow$	$AE\downarrow$
8192 pts	$\frac{(p-p^k)-\mu}{\delta+\varsigma}$	**0.042**	**96.76**	**97.29**	**0.111**
	$p-p^k$	0.046	95.25	96.08	0.132
2048 pts	$\frac{(p-p^k)-\mu}{\delta+\varsigma}$	**0.034**	**95.6**	**97.99**	**0.103**
	$p-p^k$	0.049	93.82	96.6	0.132

configurations. Consequently, we opted for $s=\{1,5,6\}$ as our experimental sampling stride set. This selection enables the effective and stable capture of multiscale neighborhood features with a reduced number of sampling points.

2) Gaussian-Based Neighborhood Feature Normalization: To verify the importance of GNFN on sparse point cloud representation, we conducted comparison experiments using the unnormalized $\triangle p = p - p^k$ and the normalized $\triangle p = ((p-p^k)-\mu)/(\delta+\varsigma)$, respectively. In these experiments, we set $s=\{1,5,6\}$. The results of the comparison are presented in Table IV.

In the first row of Table IV, "8192 pts" refers to sparsifying the input point cloud to 8192 points through random sampling. By comparing the normalization results with the nonnormalization results in the "8192 pts" row, it is evident that normalization outperforms performance. Specifically, normalization exhibits lower values in the 3DEPE metric and achieves improvements of 1.51% and 1.1% in the Acc5 and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 13:41:52 UTC from IEEE Xplore. Restrictions apply.

TABLE V
EFFECT OF THE LOGSOFT FUNCTION IN THE MNCSFP

Points	F_{NCWA}	F_{MFF}	$3DEPE\downarrow$	$Acc5(\%)\uparrow$	$Acc10(\%)\uparrow$	$AE\downarrow$
8192 pts	$Softmax$	$Softmax$	0.043	96.04	96.79	0.115
	$logsoft$	$Softmax$	0.044	96.34	97.1	0.123
	$Softmax$	$logsoft$	0.047	96.32	96.81	0.142
	$logsoft$	$logsoft$	**0.042**	**96.76**	**97.29**	**0.111**
2048 pts	$Softmax$	$Softmax$	0.051	92.77	95.89	0.126
	$logsoft$	$Softmax$	0.043	94.22	97.29	0.129
	$Softmax$	$logsoft$	0.05	93.34	96.39	0.134
	$logsoft$	$logsoft$	**0.034**	**95.6**	**97.9**	**0.103**

TABLE VI
EFFECT OF λ WHEN SAMPLING 2048 POINTS AT RANDOM

λ	$3DEPE\downarrow$	$Acc5(\%)\uparrow$	$Acc10(\%)\uparrow$	$AE\downarrow$
0.0	0.141	34.60	66.98	0.264
0.5	0.046	92.23	96.82	0.134
1.0	0.051	93.78	96.56	0.122
1.5	**0.034**	**95.65**	**97.99**	**0.103**
2.0	0.046	94.66	97.18	0.123
2.5	0.049	94.24	96.95	0.126
3.0	0.052	93.10	96.25	0.129

Acc10 metrics, respectively. Additionally, normalization also shows lower values in the AE metric. To further verify the stability of normalization operations in sparse point cloud data, the second row of Table IV presents experimental results with an input of 2048 sampling points. The results show that normalization outperforms nonnormalization in all metrics, indicating that normalization can enhance the overall performance of point feature representations. These findings confirm that our proposed GNFN operation can effectively represent the neighborhood features of sparse point clouds, thereby reducing the error and improving the accuracy of scene flow estimation.

3) Logsoft Function (Logsoft): To verify the effectiveness of the logsoft function in the NCWA and MFF modules, we conducted experiments using sparsified KITTI point cloud data. The value of s was set to $\{1, 5, 6\}$, and the sampled neighborhood clusters were represented normalized. We compared the impact of the logsoft and softmax functions on various metrics, with the experimental results summarized in Table V. In Table V, the F_{NCWA} and F_{MFF} columns represent the function types selected for the NCWA and MFF modules, respectively. The first row of Table V illustrates the impact of employing different combinations of logsoft and Softmax functions in the NCWA and MFF modules, where the number of randomly sampled points is set at 8192. It was observed that the MNCSFP model showcases superior performance when both modules incorporate the logsoft function. When the NCWA module uses logsoft and the MFF module employs Softmax, the MNCSFP model's performance is second best but surpasses the configuration where both modules use the Softmax function. This suggests that opting for the logsoft function in both modules can enhance the representation capability of features.

To further confirm the stability of the logsoft function in representing sparse point clouds, we reduced the number of random sampling points to 2048, as shown in the second row of Table V. The results indicate that when both modules use the logsoft function, the performance on various metrics is optimal, whereas when the NCWA module uses logsoft and the MFF module uses Softmax, the performance on multiple metrics is suboptimal. These findings further confirm the logsoft function's ability to more stably represent the neighborhood features of sparse point clouds, playing a significant role in improving various metrics.

4) Loss Function Hyperparameter Effects: To explore the influence of the smoothness term in the sparse point cloud scene flow estimation, we configured the number of random sampling points as 2048 and s set to $\{1, 5, 6\}$, utilizing a Gaussian-based normalization method to build neighborhood features. During the experimental phase, we started with a λ value of 0, incrementally increasing it by 0.5 to identify the optimal hyperparameter configuration. The results of the experiment are illustrated in Table VI.

In the first row of Table VI, λ is assigned a value of 0, equivalent to having the same optimization term as the FNSF [20] method. By comparing the results at $\lambda = 0$ with the FNSF (see Table I) for 2048 sampling points, it can be observed that our method achieved approximately a 10% improvement on the Acc10 metric and performed better in terms of 3DEPE and AE metrics. Therefore, the overall performance of our MNCSFP model surpasses that of the FNSF model, further confirming the effectiveness of our module design.

Further analysis of Table VI indicates that the optimal performance across all metrics was achieved at $\lambda = 1.5$. Compared to $\lambda = 0$, there was a significant improvement across all metrics. Consequently, introducing the smoothness regularization term is beneficial for the stability of sparse point cloud scene estimation and can significantly enhance the accuracy of scene flow estimation.

F. Application: Scene Flow Integration

In this section, we will leverage scene flow to integrate sparse point cloud temporal data, achieving the densification of sparse point clouds. Specifically, we are given a temporal sequence set of sparse point clouds denoted as $S = \{S_0, S_1, S_2, \ldots, S_M\}$. We will input pairs of temporally adjacent point clouds to optimize the prior model and therefore obtain a set of optimal neural network prior parameters $\Theta = \{\Theta^*_{0\to1}, \Theta^*_{1\to2}, \ldots, \Theta^*_{M-1\to M}\}$. After obtaining these parameters, we will start with the estimated scene flow $f_{0\to1} = H(p_0; \Theta^*_{0\to1})$ and iteratively estimate the long-term scene flow using a forward propagation method. The specific iterative process is as follows:

$$f_{0\to m} = f_{0\to m-1} + H\left(p_0 + f_{0\to m-1}; \Theta^*_{m-1\to m}\right). \tag{21}$$

The 3-D point $p_0 \in S_0$, with $F_{0\to m} = \{(f_{0\to m})_i\}_{i=0}^{|S_0|}$ representing the collection of per-point long-term flows from S_0 to S_m, where m is an integer between $[2, M]$. We proposed a neighborhood consistency NSFP, a continuous representation that naturally provides a continuous estimation of scene flow. Fig. 9 illustrates the densification results of the nuScene sparse point cloud data. In Fig. 9, ten point clouds are integrated into the final frame of point cloud data, achieving the densification

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 13:41:52 UTC from IEEE Xplore. Restrictions apply.

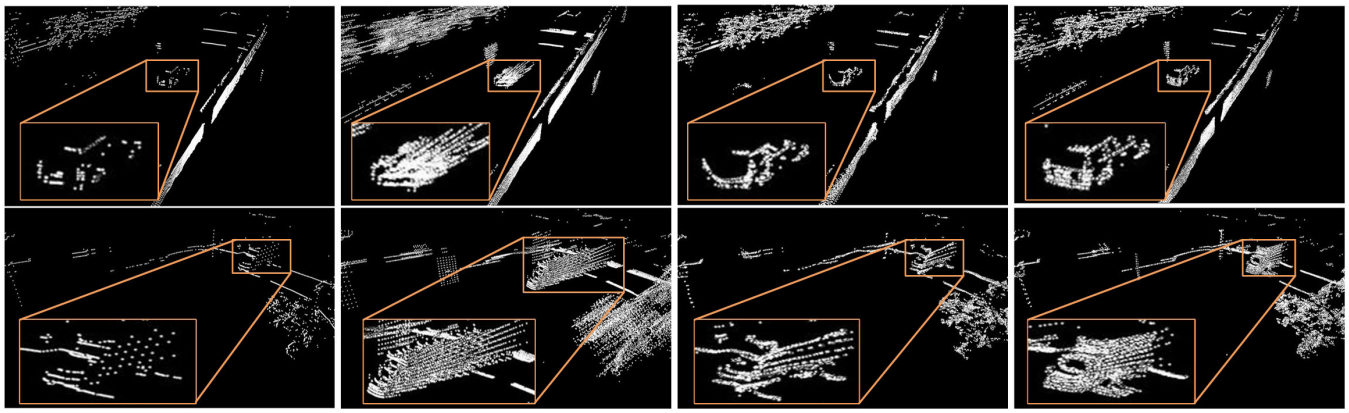


Fig. 9. Integration of nuScenes sparse point cloud data. (a) Single frame of point cloud data. (b) Cumulative results of point clouds using the rigid registration method (ICP). (c) Iterative accumulation results of the scene flow using the FNSF prior model. (d) Iterative accumulation results of sparse point cloud data using our MNCSFP method. Note, in (b), dynamic objects exhibit a smearing effect, whereas, in comparison, (c) and (d) demonstrate superior performance in point cloud densification using the prior-based methods.

of sparse point clouds. Comparing the FNSF method with our MNCSFP method in Fig. 9, it is readily apparent that our MNCSFP model excels in integrating sparse point cloud data, with moving target objects undergoing no deformation and maintaining higher integrity. This result further validates the stability and reliability of our method in real-world applications.

V. Conclusion

This article introduces a novel method for scene flow estimation that ensures neighborhood consistency, the MNCSFP model, which effectively improves the performance of scene flow estimation in sparse point clouds. In MNCSFP, we conduct a statistical analysis of the neighborhood distribution of each point in the point cloud and introduce a Gaussian-based neighborhood normalization strategy based on this analysis. Employing this normalization strategy, we propose the MNCFC module to capture the neighborhood feature representation of each point accurately. To more stably aggregate neighborhood cluster features and multiscale features, we introduce the logsoft function, which we use to design the NCWA module and the MFF module. During the optimization process, we introduce a smoothness term to ensure the consistency of scene flow neighborhoods, further increasing the accuracy of scene flow estimation. In the experimental section, we not only verified the stability of the MNCSFP model in sparse point clouds but also further confirmed the effectiveness of each module. The experimental results demonstrate that our method outperforms both learning-based and prior-based methods, fully proving the superiority and practicality of the MNCSFP model in sparse point cloud scene flow estimation.

References

[1] J. Zhao et al., "SemanticFlow: Semantic segmentation of sequential LiDAR point clouds from sparse frame annotations," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, pp. 1–11, 2023, Art. no. 5701611.

[2] D. Zhang, Z. Zheng, H. Niu, X. Wang, and X. Liu, "Fully sparse transformer 3-D detector for LiDAR point cloud," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, pp. 1–12, 2023, Art. no. 5705212.

[3] Y. Cui, Z. Fang, Z. Li, S. Li, and Y. Lin, "HyGFNet: Hybrid geometry-flow learning network for 3D single object tracking," *IEEE Trans. Intell. Vehicles*, pp. 1–12, 2024.

[4] S. Huang, Z. Gojcic, J. Huang, A. Wieser, and K. Schindler, "Dynamic 3D scene analysis by point cloud accumulation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*. Cham, Switzerland: Springer, 2022, pp. 674–690.

[5] Z. Lv, K. Kim, A. Troccoli, D. Sun, J. M. Rehg, and J. Kautz, "Learning rigidity in dynamic scenes with a moving camera for 3D motion field estimation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Sep. 2018, pp. 468–484.

[6] M. M. Karim, Z. Yin, and R. Qin, "An attention-guided multistream feature fusion network for early localization of risky traffic agents in driving videos," *IEEE Trans. Intell. Vehicles*, vol. 9, no. 1, pp. 1792–1803, Jan. 2024.

[7] Z. Yi et al., "FocusFlow: Boosting key-points optical flow estimation for autonomous driving," *IEEE Trans. Intell. Vehicles*, vol. 9, no. 1, pp. 2794–2807, Jan. 2024.

[8] X. Gu, Y. Wang, C. Wu, Y. J. Lee, and P. Wang, "HPLFlowNet: Hierarchical permutohedral lattice FlowNet for scene flow estimation on large-scale point clouds," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 3249–3258.

[9] X. Liu, C. R. Qi, and L. J. Guibas, "FlowNet3D: Learning scene flow in 3D point clouds," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 529–537.

[10] Z. Wang, S. Li, H. Howard-Jenkins, V. A. Prisacariu, and M. Chen, "FlowNet3D++: Geometric losses for deep scene flow estimation," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Mar. 2020, pp. 91–98.

[11] N. Mayer et al., "A large dataset to train convolutional networks for disparity, optical flow, and scene flow estimation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2016, pp. 4040–4048.

[12] M. Menze and A. Geiger, "Object scene flow for autonomous vehicles," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 3061–3070.

[13] M. Menze, C. Heipke, and A. Geiger, "Joint 3D estimation of vehicles and scene flow," *ISPRS Ann. Photogramm., Remote Sens. Spatial Inf. Sci.*, vol. 2, p. 427, Aug. 2015.

[14] R. Li, G. Lin, and L. Xie, "Self-point-flow: Self-supervised scene flow estimation from point clouds with optimal transport and random walk," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 15572–15581.

[15] H. Mittal, B. Okorn, and D. Held, "Just go with the flow: Self-supervised scene flow estimation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 11177–11185.

[16] G. Puy, A. Boulch, and R. Marlet, "FLOT: Scene flow on point clouds guided by optimal transport," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 527–544.

[17] Y. Kittenplon, Y. C. Eldar, and D. Raviv, "FlowStep3D: Model unrolling for self-supervised scene flow estimation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 4112–4121.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 13:41:52 UTC from IEEE Xplore. Restrictions apply.

[18] J. K. Pontes, J. Hays, and S. Lucey, "Scene flow from point clouds with or without learning," in *Proc. Int. Conf. 3D Vis. (3DV)*, Nov. 2020, pp. 261–270.
[19] X.-Q. Li, J. K. Pontes, and S. Lucey, "Neural scene flow prior," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2021, pp. 7838–7851.
[20] X. Li, J. Zheng, F. Ferroni, J. K. Pontes, and S. Lucey, "Fast neural scene flow," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 9844–9856.
[21] P. Sun et al., "Scalability in perception for autonomous driving: Waymo open dataset," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Sep. 2020, pp. 2446–2454.
[22] M. Chang et al., "Argoverse: 3D tracking and forecasting with rich maps," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Long Beach, CA, USA, Jun. 2019, pp. 8748–8757.
[23] H. Caesar et al., "nuScenes: A multimodal dataset for autonomous driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 11621–11631.
[24] D. Holz, A. E. Ichim, F. Tombari, R. B. Rusu, and S. Behnke, "Registration with the point cloud library: A modular framework for aligning in 3-D," *IEEE Robot. Autom. Mag.*, vol. 22, no. 4, pp. 110–124, Dec. 2015.
[25] J. Shin, R. Triebel, and R. Siegwart, "Unsupervised discovery of repetitive objects," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2010, pp. 5041–5046.
[26] P. W. Theiler, J. D. Wegner, and K. Schindler, "Globally consistent registration of terrestrial laser scans via graph optimization," *ISPRS J. Photogramm. Remote Sens.*, vol. 109, pp. 126–138, Nov. 2015.
[27] Q.-Y. Zhou, J. Park, and V. Koltun, "Fast global registration," in *Proc. Eur. Conf. Comput. Vis.*, Sep. 2016, pp. 766–782.
[28] X. Huang, J. Zhang, Q. Wu, L. Fan, and C. Yuan, "A coarse-to-fine algorithm for matching and registration in 3D cross-source point clouds," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 28, no. 10, pp. 2965–2977, Oct. 2018.
[29] B. Zhou, Y. Tu, Z. Jin, C. Xu, and H. Kong, "HPPLO-Net: Unsupervised LiDAR odometry using a hierarchical point-to-plane solver," *IEEE Trans. Intell. Vehicles*, vol. 9, no. 1, pp. 2727–2739, Jan. 2024.
[30] C. R. Qi, Y. Li, H. Su, and L. Guibas, "PointNet++: Deep hierarchical feature learning on point sets in a metric space," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 5100–5109.
[31] Z. Gojcic, O. Litany, A. Wieser, L. J. Guibas, and T. Birdal, "Weakly supervised learning of rigid 3D scene flow," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 5692–5703.
[32] S. A. Baur, D. J. Emmerichs, F. Moosmann, P. Pinggera, B. Ommer, and A. Geiger, "SLIM: Self-supervised LiDAR scene flow and motion segmentation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 13106–13116.
[33] I. Tishchenko, S. Lombardi, M. R. Oswald, and M. Pollefeys, "Self-supervised learning of non-rigid residual flow and ego-motion," in *Proc. Int. Conf. 3D Vis. (3DV)*, Nov. 2020, pp. 150–159.
[34] W. Wu, Z. Y. Wang, Z. Li, W. Liu, and L. Fuxin, "PointPWC-Net: Cost volume on point clouds for (self-) supervised scene flow estimation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*. Cham, Switzerland: Springer, 2020, pp. 88–107.
[35] I. Lang, D. Aiger, F. Cole, S. Avidan, and M. Rubinstein, "SCOOP: Self-supervised correspondence and optimization-based scene flow," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 5281–5290.
[36] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep learning on point sets for 3D classification and segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 652–660.
[37] G. Qian et al., "PointNeXt: Revisiting PointNet++ with improved training and scaling strategies," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2022, pp. 23192–23204.
[38] A. Boulch, "ConvPoint: Continuous convolutions for point cloud processing," *Comput. Graph.*, vol. 88, pp. 24–34, May 2020.
[39] Q. Hu et al., "RandLA-Net: Efficient semantic segmentation of large-scale point clouds," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 11108–11117.
[40] J. Yang, S. Shi, Z. Wang, H. Li, and X. Qi, "ST3D: Self-training for unsupervised domain adaptation on 3D object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 10368–10378.
[41] Z. Jin, Y. Lei, N. Akhtar, H. Li, and M. Hayat, "Deformation and correspondence aware unsupervised synthetic-to-real scene flow estimation for point clouds," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 7223–7233.
[42] P. J. Besl and N. D. McKay, "Method for registration of 3-D shapes," *Proc. SPIE*, vol. 1611, pp. 586–606, Apr. 1992.
[43] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, Jan. 2019, pp. 8026–8037.
[44] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.



Jianwang Gan is currently pursuing the Ph.D. degree with the School of Artificial Intelligence, China University of Mining and Technology (Beijing), Beijing, China.

His main research interests include image processing, point cloud semantic understanding, and machine learning.



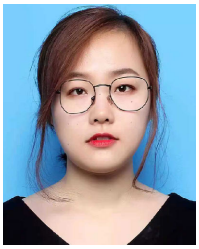
Guoying Zhang received the Ph.D. degree from the School of Computer Science, Beijing Institute of Technology, Beijing, China, in 2005.

She is currently a Professor at China University of Mining and Technology (Beijing), Beijing. Her research interests include artificial intelligence, image processes, and human–computer interaction techniques.



Juchen Zhang is currently pursuing the master's degree with the School of Artificial Intelligence, China University of Mining and Technology (Beijing), Beijing, China.

His main research directions are image processing and semantic segmentation.



Yijin Xiong received the bachelor's degree from Harbin University of Science and Technology, Harbin, China, in 2017. She is currently pursuing the Ph.D. degree with China University of Mining and Technology (Beijing), Beijing, China.

Her research focuses on multisensor fusion for intelligent connected vehicles, with research interests including autonomous vehicle localization, multisensor fusion, and vehicle–road cooperative perception.



Yongqi Gan is currently pursuing the Ph.D. degree with the School of Artificial Intelligence, China University of Mining and Technology (Beijing), Beijing, China.

His research interests include artificial intelligence and image processes.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 13:41:52 UTC from IEEE Xplore. Restrictions apply.